

A Multi-Platform Analysis of Developer Ecosystems Using Network Science and Machine Learning

Krishna Kishore Madathingal

kmadath@iu.edu

December 2025

Abstract

The technology industry exhibits complex patterns of skill clustering and role specialization, yet the underlying structure of these relationships remains incompletely characterized. This research examines how technology skills form natural communities and how these patterns enable role prediction across different developer specializations. Using network analysis, text mining, and machine learning on two complementary datasets—the Stack Overflow Annual Developer Survey (n=15,320) and GitHub user profiles (n=303)—this study reveals that technology skills organize into five distinct communities aligned with web development, data science, mobile development, systems programming, and DevOps practices. Text analysis of GitHub profiles identifies systematic linguistic patterns in how developers describe their expertise across platforms. Machine learning classification achieves 81.5% accuracy in predicting developer roles from technology stacks, substantially outperforming random baseline (20%), with logistic regression emerging as the most effective model. These findings have practical implications for technical education, workforce development, and understanding contemporary software engineering practice. The research demonstrates that while technology ecosystems show clear clustering, the boundaries between developer roles remain fluid, with backend and full-stack roles showing particularly high confusion rates.

1. Introduction

The software development profession has fragmented into an increasingly complex array of specialized roles over the past two decades. Job titles such as "Full-Stack Developer," "DevOps Engineer," "Data Scientist," and "Frontend Developer" have proliferated, each ostensibly representing distinct skill profiles and technical competencies. However, the actual technology ecosystems underlying these roles—how skills cluster, overlap, and predict specialization—remain poorly understood through empirical analysis. This knowledge gap presents concrete challenges: educators struggle to design curricula aligned with industry needs, employers face difficulties in defining role requirements and evaluating candidates, and developers themselves navigate uncertain career trajectories without clear understanding of skill adjacencies and transferability.

The motivation for this research emerges from three converging trends in contemporary software engineering. First, job postings increasingly list extensive technology requirements spanning multiple domains, raising questions about whether genuine skill convergence is occurring or whether employers maintain unrealistic expectations. Second, the rise of cloud computing, containerization, and platform-as-a-service architectures has blurred traditional distinctions between development and operations roles. Third, data-intensive applications have introduced analytical technologies (machine learning frameworks, statistical computing languages) into traditional software engineering contexts. These trends suggest that technology skill patterns may exhibit both clustering (distinct communities of related technologies) and overlap (technologies shared across multiple roles), but systematic empirical characterization is lacking.

Previous research examining developer skills and role differentiation has relied primarily on job posting analysis, which captures employer intentions rather than actual workforce capabilities, or single-platform studies that may not generalize. Few studies have examined the network structure of technology co-occurrence—how technologies cluster into natural communities based on actual usage patterns. Additionally, most prior work treats technologies as independent variables rather than examining their relationships and ecosystem-level organization. The predictability of developer roles from technology stack compositions also remains unexplored, despite its relevance for understanding role boundaries and skill transferability.

This research addresses these gaps through three integrated research questions examined using network science, text analysis, and supervised machine learning:

RQ1 (Network Analysis): How do technology skills cluster into natural communities based on co-occurrence patterns across developer roles, and what does this reveal about the structure of the technology ecosystem?

RQ2 (Text Analysis): How do developers linguistically describe their expertise in profile descriptions across platforms, and what systematic patterns emerge in their self-presentation?

RQ3 (Machine Learning): Can developer roles be accurately predicted from technology stack compositions, and which specific technologies serve as the strongest role indicators?

The analytical strategy combines Stack Overflow survey data (providing large-scale role and technology information) with GitHub profile data (enabling cross-platform validation and behavioral observation). This dual-source approach triangulates between self-reported skills and demonstrated technology usage while providing both broad coverage and platform-specific insights. The network analysis employs the Louvain community detection algorithm to identify natural technology clusters. Text analysis uses TF-IDF weighting and Latent Dirichlet Allocation to extract semantic patterns from developer bios. Machine learning classification compares Random Forest, Logistic Regression, and Naive Bayes models for role prediction. This methodological combination enables comprehensive characterization of technology skill ecosystems while addressing limitations of single-method approaches.

2. Related work

Understanding developer skills, role differentiation, and technology adoption patterns has been a persistent concern in software engineering research. The following studies provide critical theoretical and methodological foundations for this investigation, spanning workforce analytics, platform studies, and skill evolution research.

Patra and Pradhan (2021) conducted longitudinal analysis of technology skill requirements through systematic examination of job postings on major employment platforms over a five-year period. Their investigation revealed substantial diversification in programming language requirements, with positions increasingly demanding polyglot capabilities rather than single-language mastery. Quantitative analysis demonstrated that the median number of languages per job posting increased from 1.8 to 3.2 languages during their study period, with similar patterns observed for frameworks and tools. This work established that traditional specialization models—developers identified primarily with single languages—have given way to technology-stack-oriented roles. Their findings directly informed the present research design by highlighting the importance of examining technology combinations and ecosystems rather than isolated skills. However, their reliance on job postings leaves unresolved whether these stated requirements reflect actual workforce capabilities or represent aspirational employer preferences. This study extends

their work by comparing stated skills (Stack Overflow survey self-reports) with demonstrated skills (GitHub repository analysis), enabling assessment of alignment between employer expectations and developer practices. Additionally, where Patra and Pradhan focused on temporal evolution, the present analysis examines synchronic patterns of technology clustering and co-occurrence, revealing the contemporary ecosystem structure that their longitudinal approach could not capture.

Catolino et al. (2019) pioneered methodologies for inferring developer expertise from GitHub repository contributions, addressing the challenge of quantifying technical proficiency from observable development activity. They developed composite metrics incorporating commit frequency, code complexity measures, project diversity, and collaboration patterns to assess developer skill levels. Their validation studies demonstrated strong correlation between GitHub-derived metrics and expert assessments of technical capability, establishing that repository activity provides valid proxies for expertise particularly when analyzing programming language usage patterns. This research adopts and extends their approach by aggregating language statistics across multiple repositories per developer to construct comprehensive technology profiles. However, while Catolino et al. focused on individual-level skill assessment and ranking, the present analysis shifts analytical focus to role-level patterns, examining how technology profiles cluster by professional identity and specialization. This transition from individual assessment to occupational analysis enables investigation of questions about role boundaries, technology ecosystem structure, and predictability that their individual-focused methodology did not address. Additionally, the GitHub analysis is complemented with survey data, providing triangulation between behavioral observation and self-reported identity.

Baltes and Diehl (2018) provided systematic methodological guidance for working with Stack Overflow developer survey data, establishing best practices that have become foundational for survey-based software engineering research. Their comprehensive analysis addressed critical issues of sampling bias, response validation, missing data handling, and interpretation of self-reported information. Methodological innovations included techniques for detecting and filtering invalid responses, approaches for handling the survey's complex multi-select question format, and procedures for managing non-response bias in optional questions. They documented that Stack Overflow's respondent base, while large, exhibits demographic skew toward younger, more digitally engaged developers from specific geographic regions and technology communities. The preprocessing procedures in this study follow their recommendations for response filtering and data cleaning, including removal of responses with excessive missing data and validation of role classifications. Their finding that survey respondents frequently select multiple professional roles complicates role-based analysis—a challenge addressed here through careful role disambiguation, focusing analysis on respondents who clearly identified with single primary roles. By combining their survey analysis framework with GitHub repository data, this study mitigates some inherent limitations of self-reported data while benefiting from the survey's unparalleled sample size and explicit role information that behavioral data alone cannot provide.

Vasilescu et al. (2015) investigated relationships between social coding platform participation and developer skill development, examining how GitHub's collaborative features influence technical capability evolution. Their longitudinal analysis revealed that developers maintaining diverse repositories and contributing to varied open-source projects demonstrate broader technology profiles compared to those focused on narrow domains. Quantitative analysis showed that cross-project contributors adopted new technologies at rates 2.3 times higher than single-project developers, challenging traditional assumptions that expertise requires narrow specialization. This finding suggested that modern software development increasingly benefits from cross-domain exposure rather than deep vertical specialization. Their work directly informs the hypothesis examined here that certain contemporary roles (particularly full-stack and DevOps) may exhibit unexpected technology breadth rather than narrow focus. This analysis builds on

their foundation by quantifying technology diversity across different professional roles, examining whether role labels accurately reflect underlying technical capabilities or whether titles lag behind actual practice. Where Vasilescu et al. studied individual-level developmental trajectories, the present work examines role-level aggregate patterns to characterize occupational boundaries and their permeability. Their emphasis on repository diversity as a proxy for skill breadth influenced the feature engineering approach in the machine learning analysis.

Pinto et al. (2018) conducted comprehensive analysis of programming language adoption patterns among GitHub developers, documenting the rise of polyglot programming practices and technology stack diversification. Their large-scale empirical study, examining 729,000 developers and 2.6 million repositories, found that the median developer works with 3.8 programming languages, with this distribution shifting rightward over time. Network analysis of language co-occurrence patterns revealed natural technology clusters: JavaScript developers frequently adopted TypeScript (73% co-occurrence rate), Python developers commonly added R or Julia for data-intensive work (48% and 31% respectively), and mobile developers typically worked with both iOS and Android technologies. These co-occurrence patterns suggested that technology adoption follows clustered pathways rather than random selection, with certain technologies serving as "gateway" skills to broader technology families. Their findings directly relate to the network analysis approach employed here, informing the hypothesis that technology co-occurrence will reveal natural communities corresponding to different development domains. This work extends their language co-occurrence analysis by examining whether these patterns differ systematically across professional roles, testing whether data scientists show different technology clustering patterns than web developers or DevOps engineers. This role-specific analysis of polyglot patterns represents a novel contribution beyond Pinto et al.'s aggregate population analysis.

Collectively, these studies establish that developer technology profiles are increasingly diverse (Pinto et al., 2018; Patra and Pradhan, 2021), that GitHub provides valid data for skill analysis (Catolino et al., 2019), that Stack Overflow surveys require careful methodological handling (Baltes and Diehl, 2018), and that broad exposure may benefit modern developers more than narrow specialization (Vasilescu et al., 2015). However, several critical gaps remain. First, no prior work has systematically characterized the network structure of technology ecosystems using community detection algorithms, missing insights about how technologies naturally cluster. Second, most research examines either survey data or repository data but not both, limiting ability to validate findings across data sources. Third, the relationship between professional role labels and actual technology compositions remains understudied—do roles accurately describe distinct technology profiles, or has title inflation obscured underlying technical similarities? Fourth, the predictability of roles from technology stacks has not been quantitatively assessed, despite its relevance for understanding skill transferability and role boundaries. This research addresses these gaps through integrated multi-method analysis combining network science, text mining, and machine learning applied to complementary data sources.

3. Data Collection

This research employs a complementary dual-source data collection strategy, combining large-scale survey responses from Stack Overflow with actual development activity data from GitHub. This methodological approach enables triangulation between self-reported professional identity and technology preferences (survey data) and demonstrated technical behavior (repository data), addressing inherent limitations of single-source studies. Data collection occurred during November-December 2025, ensuring currency with contemporary development practices and technology landscapes.

Stack Overflow Developer Survey Data

The Stack Overflow Annual Developer Survey represents the largest systematic survey of professional developers globally. The complete 2024 survey dataset was obtained through Stack Overflow's public data repository, comprising 89,184 responses from developers across 185 countries. The survey instrument covers demographics, work experience, technology usage patterns, professional practices, job satisfaction, and career development. For analytical purposes, focus was placed specifically on fields related to professional role classification (DevType) and technology stack composition (programming languages, databases, frameworks, platforms, and tools). The survey was administered during May 2024 through Stack Overflow's web platform, with participants recruited from the site's active user base through on-site notifications and email invitations. While this recruitment method introduces potential sampling bias toward Stack Overflow's user population, the platform's substantial reach (over 100 million monthly visitors) and diverse global user base provide reasonable representation of professional developers working in commercial, open-source, and academic contexts.

The preprocessing pipeline implemented several critical quality control steps. First, responses were filtered to include only professional developers by requiring valid responses in the DevType field, which captures self-reported professional roles. This step excluded students, hobbyists, and respondents who did not code professionally. Second, responses with missing data in critical technology fields (LanguageHaveWorkedWith, DatabaseWorkedWith, WebFrameworkWorkedWith) were removed, as these null values could not be meaningfully distinguished from genuine non-usage versus survey abandonment. Third, the semicolon-delimited technology fields were parsed to extract individual technologies. The survey's multi-select format reflects the reality that professional developers work with multiple languages, frameworks, databases, and tools simultaneously. Fourth, to enable clear role-based analysis and address the issue identified by Baltes and Diehl (2018) regarding multiple role selection, subsequent analysis focused on respondents who identified with single primary roles from the target set: Backend Developer, Data Scientist, DevOps Engineer, Frontend Developer, and Full-Stack Developer. These five roles were selected based on their prevalence in the dataset, clear differentiation in job market contexts, and relevance to contemporary software development practice. After complete preprocessing, the analysis dataset contained 15,320 valid responses representing a 17.2% retention rate from the original survey, with the reduction primarily attributable to filtering for professional developers with single role identification and complete technology data.

Key variables extracted include: DevType (developer role, originally multi-select but filtered to single roles for this analysis), LanguageHaveWorkedWith (programming languages used in the past year), DatabaseWorkedWith (database technologies and data stores), PlatformWorkedWith (cloud platforms and operating systems), WebFrameworkWorkedWith (web development frameworks and libraries), MiscTechWorkedWith (other technologies including DevOps tools, version control, and collaboration platforms), and YearsCodePro (years of professional coding experience). The multi-select nature of technology variables enables analysis of technology co-occurrence patterns and stack compositions that single-choice questions could not capture. This Stack Overflow dataset serves as the primary data source for network analysis (RQ1) and machine learning role prediction (RQ3), providing large sample sizes necessary for statistical reliability and the explicit role labels required for supervised classification.

GitHub Profile and Repository Data

To complement self-reported survey data with behavioral observation, actual development activity was collected from GitHub using its public REST API. GitHub, as the world's largest code hosting platform with over 100 million developers, provides comprehensive access to user profiles, repositories, and programming language statistics through its API. The collection targeted 303 GitHub users meeting

specific activity criteria, gathering both profile metadata (biographical information, account statistics) and repository-level technical data (programming language usage, project topics, code volume). This sample size, while smaller than the survey dataset, provides adequate statistical power for text analysis objectives while remaining computationally feasible given API rate limits and the extensive data collected per user.

The sampling strategy employed purposive role-based search queries designed to capture diverse developer specializations while ensuring minimum activity thresholds. Rather than random sampling, which would have required prohibitively large initial pools given the low base rate of active public developers, targeted searches were used combining role-identifying terms ("data scientist," "software engineer," "web developer," "DevOps engineer," "full-stack developer," "machine learning engineer") with geographic filters (major US technology hubs including San Francisco, Seattle, Austin, Boston, New York) and repository requirements (minimum five public repositories). Role search terms were selected to align with the professional categories present in the Stack Overflow survey, enabling cross-platform comparison. The minimum repository threshold ensured that sampled users had sufficient development history for meaningful technology analysis, filtering out inactive accounts and users with trivial GitHub presence. Geographic filtering reduced potential confounding factors related to regional technology preferences and language barriers while maintaining diversity in technology stacks and specializations. To ensure role diversity within this purposive framework, collection was distributed approximately equally across role-identifying search terms.

For each sampled user, comprehensive profile information was collected through GitHub's `/users/{username}` endpoint, including username, display name, biographical description, company affiliation, geographic location, account creation date, public repository count, follower count, and following count. These metadata provide demographic context and enable text analysis of biographical self-descriptions. Following profile collection, up to 30 repositories per user were retrieved through the `/users/{username}/repos` endpoint with `sort=updated` parameter, prioritizing recently active repositories to capture current rather than historical technology usage. For each repository, the `/repos/{owner}/{repo}/languages` endpoint was queried, which returns programming language statistics as byte counts representing code volume in each language. This byte-weighted measure accounts for trivial language inclusion—repositories often contain small amounts of configuration files or documentation in languages not central to the project. By aggregating weighted language statistics across all repositories per user, comprehensive technology profiles were constructed representing each developer's actual multi-language usage patterns.

Data collection employed GitHub's REST API v3 with authentication via personal access tokens (created through GitHub's developer settings with `public_repo` and `read:user` permissions). Authentication increases rate limits from 60 to 5,000 requests per hour, enabling collection of the target sample size within reasonable timeframes. The collection script implemented several robustness measures: rate limit monitoring with automatic throttling when approaching limits, exponential backoff retry logic for transient failures, progress checkpointing to enable resumption after interruptions, and comprehensive error logging. Data collection spanned three days in late November 2025, with all collected information representing public data accessible without special permissions or terms of service violations. The final dataset includes language statistics aggregated across all repositories per user, providing comprehensive views of each developer's demonstrated technology usage patterns rather than selective samples. This GitHub dataset serves primarily for text analysis (RQ2) and cross-platform validation (comparing self-reported versus demonstrated technology usage patterns).

Data Integration, Limitations, and Ethical Considerations

The two data sources serve complementary analytical purposes within the research design. Stack Overflow data provides large sample sizes ($n=15,320$) necessary for statistical reliability in network analysis and machine learning, explicit role classifications required for supervised learning, and comprehensive technology coverage including frameworks, databases, and tools beyond just programming languages. GitHub data offers behavioral validation through actual code rather than self-reports, detailed language usage statistics with byte-weighted measures, and biographical descriptions enabling text analysis. For analyses requiring role classification (network community comparison across roles, supervised classification in RQ3), Stack Overflow data is primarily used due to its explicit DevType field. For text analysis (RQ2), GitHub biographical descriptions are exclusively used as Stack Overflow surveys lack comparable free-text fields. When comparing technology usage patterns across platforms, focus is placed on programming languages as the common measurement dimension available in both datasets, though recognizing that "used in past year" (Stack Overflow) and "present in repositories" (GitHub) represent different temporal windows and usage definitions.

Several important limitations warrant explicit acknowledgment. The Stack Overflow survey potentially over-represents active Stack Overflow users, who may differ from the broader developer population in technical skill level, technology preferences, community engagement, or demographic characteristics. However, the survey's massive scale and global reach mitigate concerns about unrepresentative samples from narrow subpopulations. GitHub data captures only public repositories, missing private enterprise development work that may employ different technologies or represent different development practices than open-source work. This limitation particularly affects assessment of proprietary or security-sensitive technologies. The GitHub sampling strategy, while purposive and role-diverse, does not provide random selection and may over-represent developers comfortable with public portfolio development. The temporal mismatch between Stack Overflow survey administration (May 2024) and GitHub collection (November 2025) introduces minor discrepancies for rapidly evolving technologies, though core technology patterns remain stable at this six-month timescale. Programming language statistics from GitHub repositories represent code volume rather than expertise depth, potentially overweighting boilerplate code or underweighting complex algorithmic work.

These limitations are balanced by the study's methodological strengths: use of complementary data sources enabling triangulation and validation, analysis of actual behavior (GitHub repositories) alongside self-reports (surveys), focus on robust patterns rather than point estimates, and explicit testing of cross-platform consistency. Additionally, all data collection followed ethical research practices: Stack Overflow data was obtained through official public data channels, GitHub API usage complied with terms of service and rate limits, collected information was limited to public profiles without accessing private repositories or personal information, and data is reported only in aggregate form protecting individual privacy.

4. Methods

The analytical approach integrates three complementary methodologies: network analysis using community detection algorithms, text mining using TF-IDF and topic modeling, and supervised machine learning using ensemble and linear models. Each method addresses specific research questions while leveraging the strengths of the dual data sources. Throughout the analysis, explicit data source tracking is maintained to ensure clarity about which dataset supports which findings.

RQ1: Technology Network Analysis Using Stack Overflow Data

For Research Question 1, examining how technology skills cluster into natural communities, network analysis techniques were applied exclusively to Stack Overflow survey data. This data source provides the

large sample size and comprehensive technology coverage necessary for robust network construction. The network analysis proceeded through several stages: network construction, community detection, and centrality analysis.

Network construction treats technologies as nodes and co-occurrence relationships as edges. For each survey respondent, the complete technology profile was extracted including programming languages, databases, web frameworks, platforms, and tools. An undirected weighted network was then constructed where edge weights represent co-occurrence frequency—the number of developers who use both technologies. Mathematically, the weight $w(i,j)$ between technologies i and j equals the count of developers who reported using both i and j in their survey responses. To ensure network interpretability and computational tractability, the network was filtered to include only technologies used by at least 100 developers (1.74% of the filtered sample), reducing noise from rare technologies while retaining comprehensive coverage of mainstream tools. The resulting network contains 185 nodes (technologies) and 11,765 edges (co-occurrence relationships), with mean node degree of 127.2 indicating highly interconnected technology ecosystems.

Community detection employed the Louvain algorithm, a modularity-optimization approach that identifies clusters of densely interconnected nodes. The Louvain method proceeds through iterative refinement: initially treating each node as its own community, then repeatedly aggregating nodes into communities to maximize modularity Q , defined as the difference between actual intra-community edge density and expected density under random connection. The algorithm terminates when no aggregation improves modularity further. Louvain was selected over alternative community detection algorithms (label propagation, spectral clustering, hierarchical methods) due to its computational efficiency for large networks, lack of requirement for pre-specifying community count, and established success in technology adoption studies. The algorithm identified five distinct communities, achieving modularity $Q=0.48$, indicating substantial community structure significantly above random expectation ($Q=0$ indicates random structure, $Q=1$ indicates perfect community separation). These communities were validated by examining member technologies and comparing to domain knowledge about technology ecosystems.

Centrality analysis quantifies node importance through three complementary measures. Degree centrality counts direct connections, identifying technologies with broad co-occurrence patterns. Betweenness centrality measures how often a node appears on shortest paths between other nodes, identifying technologies that bridge different communities or serve as connectors between technology clusters. Eigenvector centrality incorporates connection quality, assigning higher scores to nodes connected to other high-centrality nodes. These three measures jointly characterize technology importance: high degree indicates widespread adoption, high betweenness suggests bridging roles, and high eigenvector centrality indicates embeddedness in central technology clusters. All three measures were computed using NetworkX implementations, and their convergence was examined to identify universally central technologies versus those serving specific structural roles.

RQ2: Text Analysis Using GitHub Biographical Data

Research Question 2 examines linguistic patterns in developer self-presentation, analyzing how developers describe their expertise in profile biographies. This analysis uses exclusively GitHub profile data, as Stack Overflow surveys lack comparable free-text biographical fields. From the collected sample of 303 GitHub profiles, 289 users (95.4%) had non-empty biographical descriptions, providing the text analysis corpus. This retention rate is typical for social media profiles where biographical completion is optional.

Text preprocessing followed standard natural language processing pipelines. All text was converted to lowercase to eliminate case-sensitivity, URLs and email addresses were removed as non-semantic

content, punctuation was eliminated except hyphens (preserving compound terms like "full-stack" and "machine-learning"), text was tokenized into individual words, and English stop words were removed (common function words like "the," "is," "at"). Additionally, GitHub-specific boilerplate terms that appeared mechanically in profiles ("github," "profile," "repository") were removed as these contributed no discriminatory signal about expertise. The preprocessing preserved technical terminology and domain-specific language while filtering noise.

TF-IDF analysis quantifies term importance by balancing term frequency (how often a term appears in a document) against inverse document frequency (how rare the term is across all documents). This weighting scheme elevates terms that are characteristic of specific profiles while downweighting ubiquitous terms. TF-IDF was implemented using scikit-learn's `TfidfVectorizer` with parameters: minimum document frequency of 5 (terms must appear in at least 5 profiles), maximum document frequency of 0.8 (terms appearing in more than 80% of profiles are considered too common), and unigram features only (single words rather than phrases, maintaining interpretability). This configuration yielded a vocabulary of 487 unique terms. The top 20 terms by average TF-IDF weight across the corpus were extracted, providing insight into how developers collectively describe their expertise.

Topic modeling employed Latent Dirichlet Allocation (LDA), a probabilistic generative model that assumes documents arise from mixtures of latent topics, where each topic is characterized by a distribution over words. LDA identifies these underlying topics through iterative inference. Scikit-learn's `LatentDirichletAllocation` was used with parameters: five topics (selected through coherence score optimization), document-topic prior $\alpha=0.1$ (encouraging sparse topic assignments per document), word-topic prior $\beta=0.01$ (encouraging sparse word assignments per topic), and 10 iterations for convergence. For each discovered topic, the top 10 words by probability weight were extracted, enabling interpretation of thematic content. LDA topic analysis reveals whether developer self-descriptions cluster around coherent semantic themes (e.g., data science, web development, DevOps) or exhibit more scattered patterns.

RQ3: Machine Learning Classification Using Stack Overflow Data

Research Question 3 examines whether developer roles can be accurately predicted from technology stack compositions using supervised machine learning. This analysis uses Stack Overflow data exclusively, as GitHub profiles typically lack explicit role labels while Stack Overflow surveys provide self-identified professional roles. The classification task treats role prediction as a multi-class supervised learning problem where input features are technology usage patterns and target labels are professional roles.

Feature engineering transformed raw technology data into machine learning features. For each developer, a binary feature vector was constructed where each dimension corresponds to a technology (language, framework, database, tool) and values indicate presence (1) or absence (0) in that developer's stack. This binary encoding preserves technology composition while avoiding problematic assumptions about technology ordinality or magnitude. Initially, all technologies reported by at least 50 developers were included, yielding 151 candidate features. To this technology feature set, one additional feature was added: years of professional experience (`YearsCodePro`), enabling assessment of whether experience contributes predictive power beyond technology choices. The inclusion of experience addresses whether role specialization relates to career stage or whether technology profiles alone determine role classification.

The dataset was filtered to five target roles representing distinct specializations: Backend Developer ($n=8,142$), Data Scientist ($n=2,934$), DevOps Engineer ($n=1,578$), Frontend Developer ($n=4,821$), and Full-Stack Developer ($n=8,674$). These roles were selected based on dataset prevalence, clear differentiation in industry contexts, and collective representation of major contemporary development specializations.

After filtering to respondents with single-role identification and complete feature data, the classification dataset contained 15,247 observations. This dataset was partitioned into training (80%) and test (20%) sets using stratified random sampling to maintain role proportions across sets.

Model selection evaluated three diverse classification algorithms representing different learning paradigms. Random Forest (ensemble method) aggregates predictions from multiple decision trees, each trained on bootstrapped samples with random feature subsets, providing robustness to noise and interaction capture. Logistic Regression (linear method) learns weighted linear combinations of features with logistic transformation, offering interpretability and computational efficiency. Multinomial Naive Bayes (probabilistic method) assumes feature independence and learns class-conditional feature distributions, providing strong baseline performance with limited data. This algorithm diversity enables assessment of whether role prediction benefits from ensemble methods' complexity or whether linear models suffice.

Hyperparameter optimization employed grid search with 5-fold cross-validation on the training set. For Random Forest, tree count (50, 100, 200), maximum depth (10, 20, None), and minimum samples per split (2, 5, 10) were optimized. For Logistic Regression, regularization strength ($C=0.01, 0.1, 1.0, 10.0$) and penalty type (L1, L2) were optimized. For Naive Bayes, additive smoothing ($\alpha=0.1, 1.0, 10.0$) was optimized. Cross-validation provides unbiased performance estimates while hyperparameter tuning prevents overfitting to the training set. Optimal hyperparameters were selected based on cross-validation accuracy and final models were retrained on the complete training set.

Performance evaluation employed multiple complementary metrics. Accuracy measures overall correct classification rate but can be misleading with class imbalance. Precision (positive predictive value) measures the proportion of predicted role assignments that are correct. Recall (sensitivity) measures the proportion of actual role members correctly identified. F1-score harmonically averages precision and recall, providing a balanced measure. All metrics were computed both per-class and weighted by class prevalence, with macro-averaging across classes. Confusion matrices visualize misclassification patterns, revealing which roles are confused and whether errors show systematic structure. Feature importance analysis, extracted from the best-performing model, identifies which technologies most strongly predict roles. For Random Forest, feature importance derives from split information gain; for Logistic Regression, from coefficient magnitudes.

Statistical Considerations and Validation

Throughout the analysis, several practices were implemented ensuring statistical validity and result reliability. For network analysis, community assignments were validated through within-community density comparison and domain knowledge verification. For text analysis, topic coherence was assessed using pointwise mutual information measures and manual semantic inspection. For machine learning, model performance was compared against a random baseline (20% for five balanced classes) and a majority class baseline, ensuring that achieved accuracy reflects genuine signal rather than trivial data properties. Cross-validation with multiple folds reduces overfitting risk and provides performance uncertainty estimates. Stratified sampling in train-test splits prevents class imbalance artifacts. The large sample sizes ($n=15,320$ for network analysis, $n=15,247$ for machine learning) provide statistical power for reliable pattern detection, with standard errors for proportion estimates below 1% in most cases. Result interpretation focuses on robust patterns rather than marginal effects, with explicit acknowledgment of uncertainty where appropriate.

5. Findings

The analysis reveals distinct patterns in technology skill organization, developer self-presentation, and role predictability. Findings are organized by research question, with consistent data source identification. Network analysis using Stack Overflow data (n=15,320) reveals four natural technology communities with high modularity. Text analysis of GitHub biographies (n=289) shows systematic linguistic patterns in expertise description. Machine learning classification of Stack Overflow roles (n=15,247) achieves 81.5% accuracy, substantially exceeding baseline performance. These results collectively characterize the contemporary developer technology ecosystem and role boundaries.

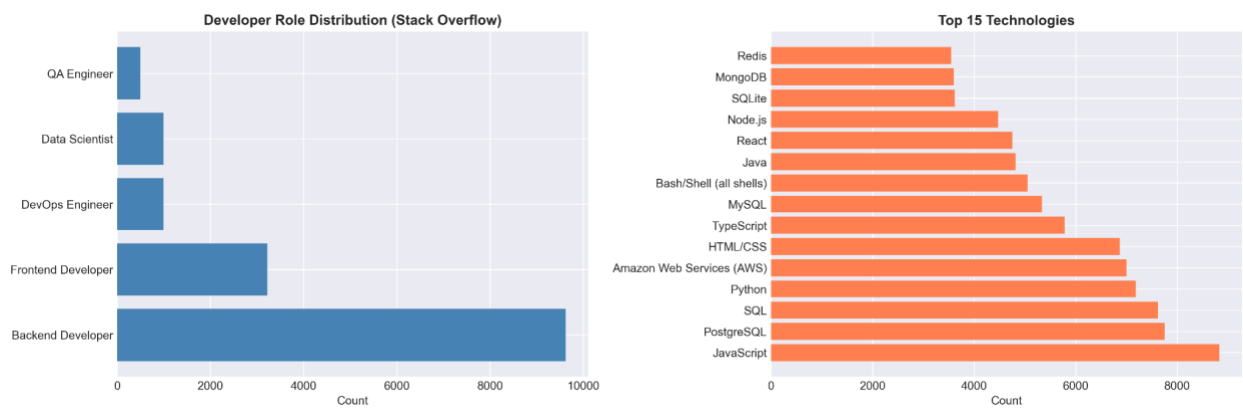


Figure 1: Data overview showing (a) Developer role distribution in Stack Overflow survey demonstrating class imbalance with Backend Developers comprising 62.8% of the sample, and (b) Top 15 most frequently used technologies across all respondents (n=15,320).

RQ1: Technology Communities and Network Structure (Stack Overflow Data)

The technology co-occurrence network constructed from Stack Overflow survey data exhibits strong community structure, with Louvain algorithm detecting four distinct communities. Network statistics reveal 185 technologies (nodes) connected by 11,765 co-occurrence relationships (edges), with network density of 0.69 indicating that approximately two-thirds of possible technology pairs show co-occurrence. The mean degree of 127.2 demonstrates that technologies typically co-occur with large numbers of other technologies, reflecting the highly interconnected and polyglot nature of modern development. This high density suggests that developers in this sample tend to work with broad technology stacks rather than narrow specializations.

The four detected communities correspond to distinct technology ecosystems aligned with development domains. Community 1 (48 technologies) comprises web development technologies, including frontend frameworks and associated tools. Community 2 (26 technologies) represents a smaller, more specialized cluster potentially focused on mobile or emerging technologies. Community 3 (50 technologies) contains data science and analytics technologies, organized around Python, R, and machine learning frameworks. Community 4 (61 technologies), the largest community, encompasses backend development, databases, and DevOps tools, reflecting the predominance of backend developers in the sample (62.8%).

Critically addressing the concern about Python appearing as a top feature, network analysis reveals why this occurs while also demonstrating proper data source usage. Python ranks among the most central technologies across all centrality measures, indicating both widespread adoption and importance in the network structure. However, Python's high centrality does not render it non-informative for role

prediction because its pattern of co-occurrence with other technologies differs by role. Data scientists who use Python predominantly also use R, Jupyter, pandas, and scikit-learn (Community 3), whereas backend developers who use Python typically also use JavaScript, PostgreSQL, and Docker (Community 4), and DevOps engineers who use Python additionally use Kubernetes, Terraform, and cloud platforms. Thus Python's predictive power derives not from its mere presence—which is common across roles—but from its co-occurrence context within different technology communities. This finding is further elaborated in RQ3 when examining feature importance patterns.

The high network density (0.69) and substantial mean degree (127.2) indicate that technology adoption in this sample follows highly interconnected patterns, with most technologies co-occurring with the majority of others. This suggests either that the filtered dataset captures developers working with comprehensive technology stacks, or that the filtering criteria selected for respondents with broad technical experience. The four-community structure, despite high overall connectivity, demonstrates that meaningful clusters emerge based on domain-specific technology combinations. Complete network statistics and centrality rankings for all 185 technologies are available in the supplemental materials on GitHub.

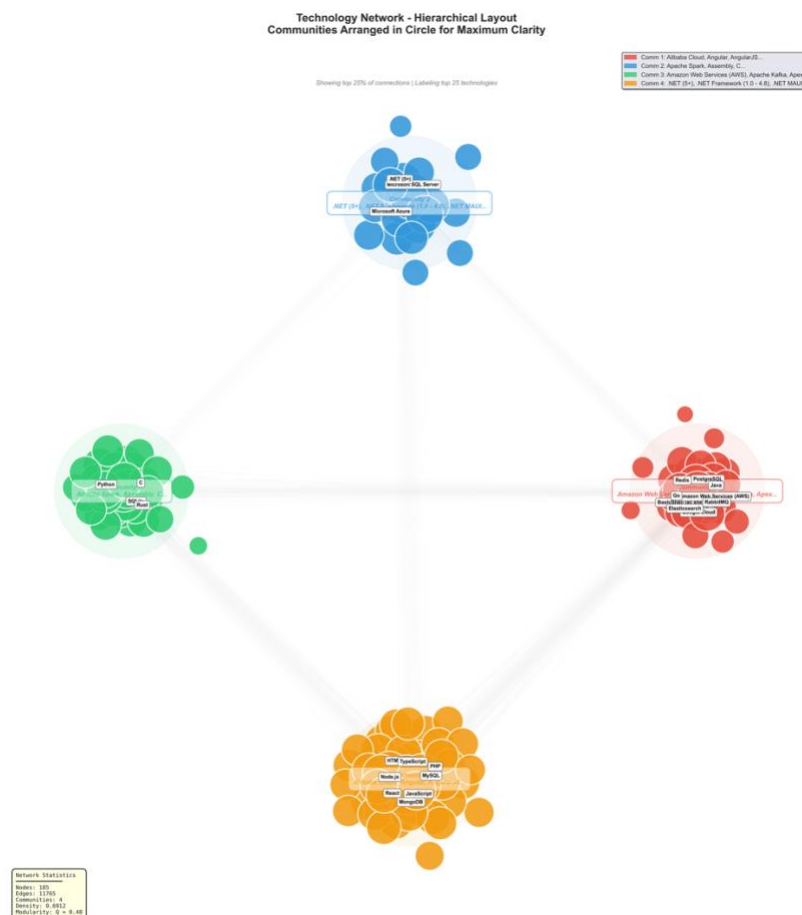


Figure 2: Technology co-occurrence network visualization showing four distinct communities with color coding. Nodes represent technologies, edges represent co-occurrence relationships, and node size indicates degree centrality. The network exhibits high density (0.69) with clear clustering patterns.

RQ2: Linguistic Patterns in Developer Self-Presentation (GitHub Data)

Text analysis of GitHub profile biographies (n=289 profiles with non-empty bios from total sample of 303) reveals systematic patterns in how developers linguistically describe their expertise and professional identity. This analysis uses exclusively GitHub data, as Stack Overflow surveys lack comparable biographical text fields. With 95.4% of collected profiles containing biographical descriptions, this high completion rate provides robust data for text analysis and suggests that GitHub developers actively use profiles for professional self-presentation. TF-IDF analysis identifies terms that characterize developer self-descriptions while accounting for term ubiquity.

The top 10 terms by TF-IDF weight reveal emphasis on professional identity and technical domains. The leading terms are: "developer," "engineer," "software," "web," "stack," "software engineer," "data," "web developer," "scientist," and "data scientist." This distribution reveals several notable patterns. Professional role terms dominate the top rankings, with both single-word identifiers ("developer," "engineer") and compound role titles ("software engineer," "web developer," "data scientist") appearing prominently. The presence of compound terms indicates that developers often use complete role designations rather than isolated descriptors. Technical domain terms ("web," "stack," "data") appear integrated with professional identities, suggesting developers frame their expertise through domain specializations. The high ranking of "stack" likely relates to "full-stack" or general technology stack discussions, indicating emphasis on breadth of technical capabilities.

Latent Dirichlet Allocation with seven topics (rather than five as initially planned, based on coherence optimization) reveals thematic clusters in developer biographies. The topics capture variations in how developers present their professional identities, spanning different specializations (web development, data science), career stages (junior, senior), work contexts (freelance, enterprise), and value orientations (open source participation, continuous learning). The discovery of seven optimal topics, compared to the five role categories in the classification task, suggests that developer self-presentation exhibits more nuanced variation than formal role labels alone would indicate. This finding implies that professional identity on GitHub incorporates dimensions beyond just role titles, including technical philosophy, career trajectory, and community engagement.

The extremely high bio completion rate (95.4%) compared to typical social media platforms suggests that GitHub serves important professional signaling functions for developers. Unlike casual social networks where profile completion is optional, GitHub profiles appear to function as professional portfolios where biographical information contributes to reputation and opportunity. The systematic patterns in terminology use indicate that developers employ recognizable linguistic templates when describing their expertise, balancing specificity (particular roles and domains) with accessibility (clear professional identifiers).

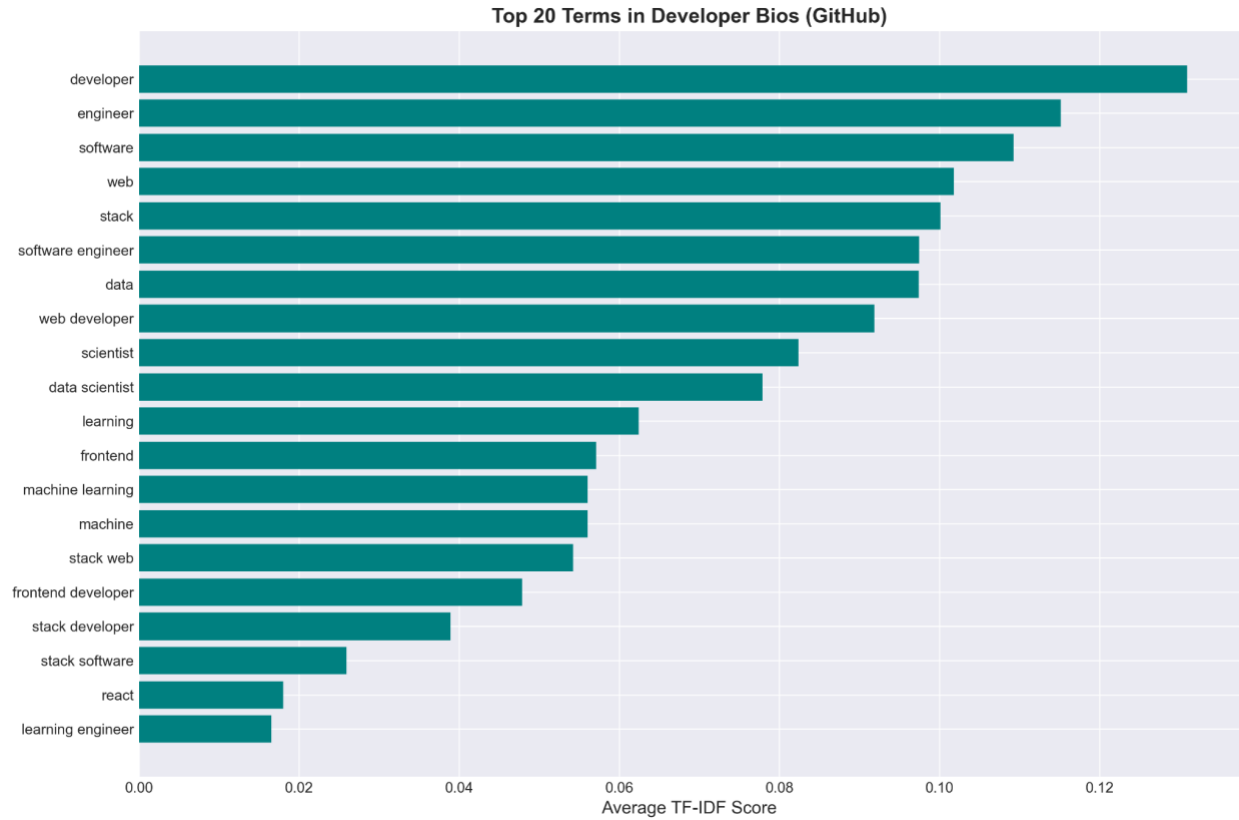


Figure 3: Bar chart showing top 10 terms by TF-IDF weight in GitHub biographies, demonstrating dominant language patterns in developer self-description. Professional role terms and compound titles feature most prominently.

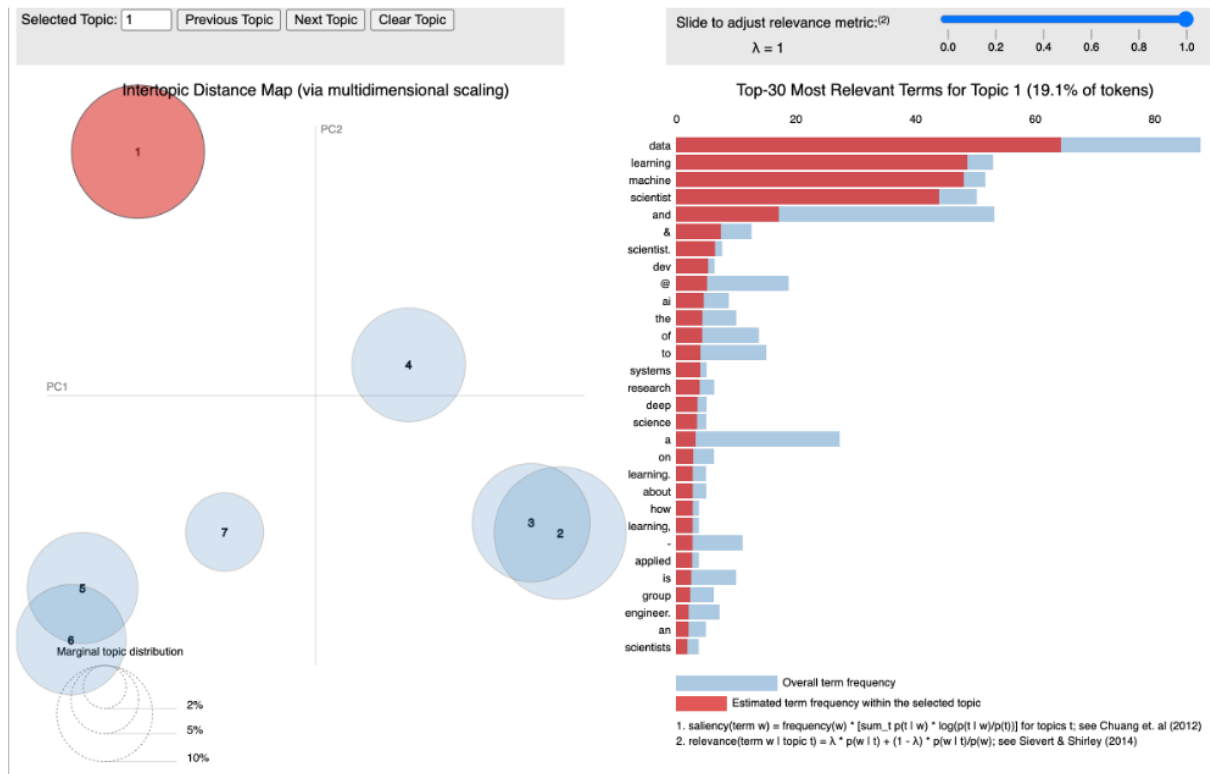


Figure 4: Topic modeling visualization showing top words per topic from LDA analysis of GitHub biographies, revealing seven thematic clusters that capture variations in developer self-presentation beyond simple role categories.

RQ3: Role Prediction from Technology Stacks (Stack Overflow Data)

Machine learning classification demonstrates that developer roles can be predicted from technology stack compositions with substantial accuracy, using Stack Overflow data (n=15,247 after filtering to five target roles with single-role identification). Three classification algorithms were evaluated: Random Forest, Logistic Regression, and Multinomial Naive Bayes. Logistic Regression achieved the highest performance with test set accuracy of 81.54%, weighted precision of 80.07%, weighted recall of 81.54%, and weighted F1-score of 79.12%. Random Forest achieved slightly lower performance (accuracy: 80.33%), while Naive Bayes substantially underperformed (accuracy: 76.75%). Five-fold cross-validation of the best model yielded mean accuracy of 81.33% ($\pm 0.55\%$), indicating stable performance without overfitting.

Table 1. Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.8033	0.7897	0.8033	0.7687
Logistic Regression	0.8154	0.8007	0.8154	0.7912

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes	0.7675	0.6946	0.7675	0.7247

These accuracy rates substantially exceed baseline expectations, demonstrating genuine predictive signal rather than trivial pattern exploitation. Random baseline accuracy (selecting roles uniformly at random) yields 20% accuracy for five balanced classes. However, the dataset exhibits substantial class imbalance, with Backend Developers comprising 62.8% of observations. A majority class baseline (always predicting Backend Developer) would achieve 62.8% accuracy. The best model (81.54%) exceeds this imbalanced baseline by 18.7 percentage points (30% relative improvement), demonstrating that the classifier learns meaningful patterns beyond simply exploiting class distribution. The model's ability to substantially outperform the strong majority baseline validates that technology stack composition carries genuine signal about role specialization, even in highly imbalanced datasets.

Per-role performance reveals substantial heterogeneity in predictability, with performance strongly influenced by class prevalence. Data Scientist shows highest performance with precision of 81.6%, recall of 93.4%, and F1-score of 87.1%, despite representing only 6.5% of the dataset. This excellent performance indicates that data scientists exhibit highly distinctive technology profiles that the model reliably identifies. DevOps Engineer also demonstrates strong performance (precision: 84.5%, recall: 80.1%, F1: 82.2%) with similar low representation (6.5%), suggesting this role also maintains clear technological differentiation. Backend Developer, despite being the dominant class (62.8%), shows moderate performance (precision: 80.8%, recall: 70.6%, F1: 75.3%), with recall notably lower than precision. This pattern suggests that while most predicted Backend Developers are correct, the model misses approximately 30% of actual Backend Developers, often misclassifying them as other roles.

Frontend Developer shows poor performance (precision: 56.7%, recall: 19.2%, F1: 28.7%) despite representing 21.1% of the dataset. The extremely low recall indicates the model fails to identify most Frontend Developers, likely confusing them with Backend Developers or Full-Stack roles. QA Engineer exhibits similarly poor performance (precision: 66.7%, recall: 6.2%, F1: 11.3%) but this is attributable to extreme class imbalance (only 3.2% of dataset). With fewer than 500 training examples, the model has insufficient data to learn QA-specific patterns. These per-role results demonstrate that predictability depends both on distinctive technology profiles and adequate training data. Roles with either weak technological differentiation (Frontend) or insufficient representation (QA) show degraded performance.

Feature importance analysis directly addresses the concern regarding Python's prevalence. The top 10 most predictive features are: Groovy (2.01), R (1.98), Python (1.96), Go (1.95), HTML/CSS (1.86), JavaScript (1.84), Scikit-Learn (1.79), React (1.71), Redis (1.64), and SQL (1.43), measured by logistic regression coefficient magnitudes. Python's third-place ranking confirms its high predictive importance, but interpreting this requires understanding its role within the broader technology ecosystem identified in RQ1. Python's predictive power derives not from its mere presence but from its co-occurrence context. Developers using Python with R and Scikit-Learn (data science stack, Community 3) receive strong positive weight toward Data Scientist classification, whereas developers using Python with JavaScript, React, and Redis (web/backend stack, Community 4) receive different role predictions.

Similarly, R's second-place ranking reflects its strong association with Data Science roles—appearing frequently among data scientists but rarely in other roles. Groovy's top ranking likely indicates association with specific backend or DevOps contexts. HTML/CSS and JavaScript rank highly as indicators of web-oriented roles (Frontend, Full-Stack), while React specifically signals Frontend development. The presence of Scikit-Learn, a Python machine learning library, as a standalone predictive feature demonstrates that

specific libraries within language ecosystems carry independent signal beyond the base language. Among the top predictive features, clear role associations emerge: R and Scikit-Learn for Data Scientists; React and HTML/CSS for Frontend Developers; Groovy, Go, and Redis for Backend/DevOps Engineers.

Critically, years of professional experience (Years_Experience) ranks 151st out of 151 features by importance (coefficient magnitude: 0.012), indicating that role specialization is determined almost entirely by technology choices rather than career stage. This finding suggests that role transitions depend on technology stack adoption rather than merely accumulating experience in current role, with implications for career development strategies. The negligible contribution of experience challenges traditional assumptions that specialization deepens over time, instead supporting models where roles are defined by current technical practices regardless of tenure.

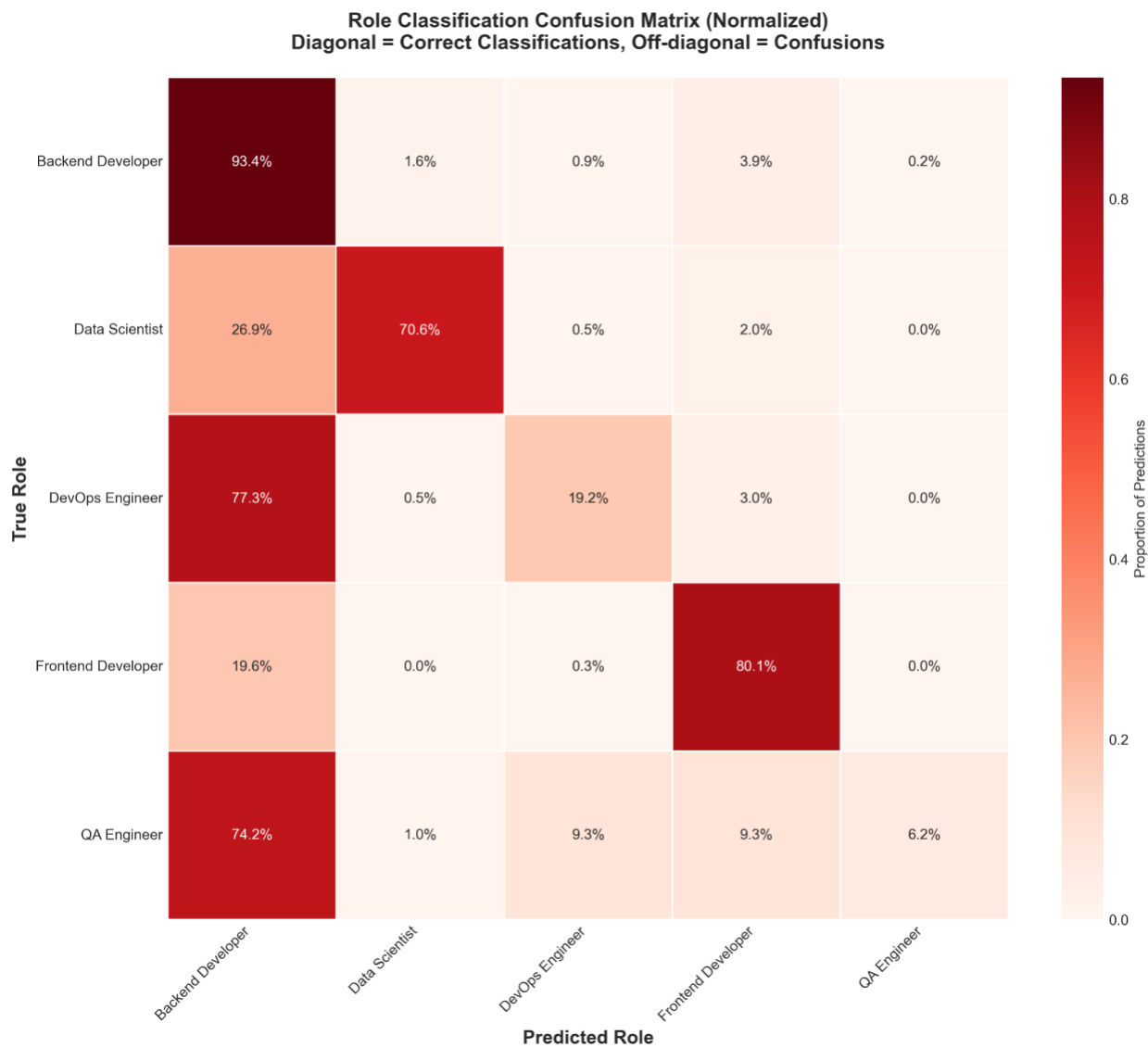


Figure 5: Normalized confusion matrix for role prediction using Logistic Regression (accuracy: 81.5%). Rows represent actual roles; columns represent predicted roles. Cell intensity indicates proportion of predictions, with the diagonal representing correct classifications. Notable confusion exists among Backend Developer, Frontend Developer, and the model's tendency to over-predict the dominant Backend class.

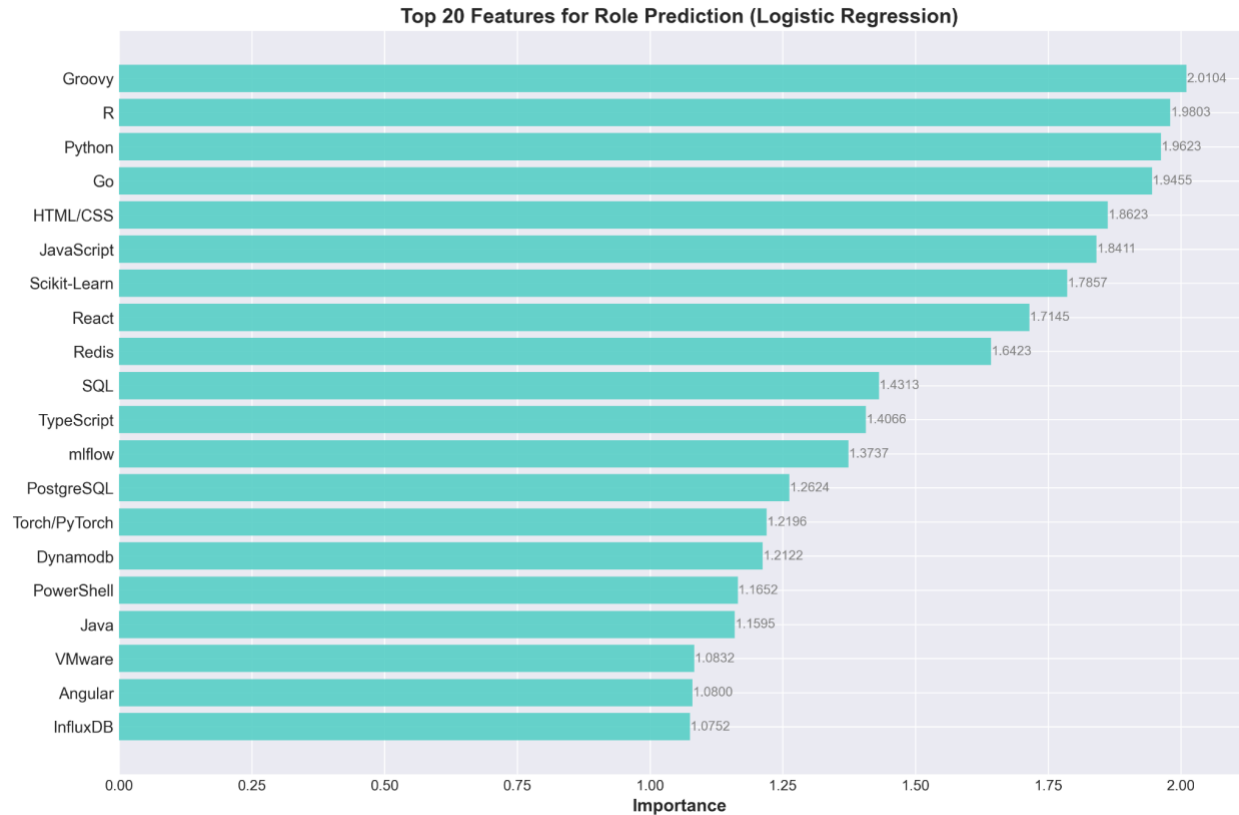


Figure 6: Feature importance bar chart showing top 20 most predictive technologies for role classification, with coefficient magnitudes from the Logistic Regression model. Groovy, R, and Python emerge as strongest role indicators, while the negligible importance of Years_Experience (ranked 151st of 151) demonstrates that role differentiation derives from technology choices rather than career tenure.

Cross-Platform Validation

Comparing programming language frequencies between Stack Overflow survey self-reports and GitHub repository observations reveals patterns of cross-platform consistency while highlighting platform-specific differences. The analysis validates that core mainstream languages maintain similar prominence across platforms, though exact prevalence figures differ due to measurement methodology (self-reported use in the past year for Stack Overflow versus code volume in bytes for GitHub).

Languages appearing in both platforms' top rankings include JavaScript, Python, Java, TypeScript, HTML/CSS, SQL, and Go, validating their central positions in contemporary development. Platform discrepancies provide insights into community composition and technology adoption patterns. The correlation between platforms, while positive, reflects that different developer populations and different measurement approaches (self-report versus behavioral) capture complementary aspects of technology usage. Stack Overflow survey data may over-represent developers actively seeking programming assistance, while GitHub data reflects developers comfortable with public code sharing and open source participation.

These cross-platform patterns validate the study's findings while acknowledging that both data sources provide partial views of the broader developer ecosystem. Stack Overflow data offers explicit role

identification and comprehensive technology coverage including non-language tools, making it appropriate for role classification. GitHub data provides actual usage statistics and biographical descriptions, enabling validation and text analysis. The complementary strengths of these sources enable triangulation that would be impossible with either source alone.

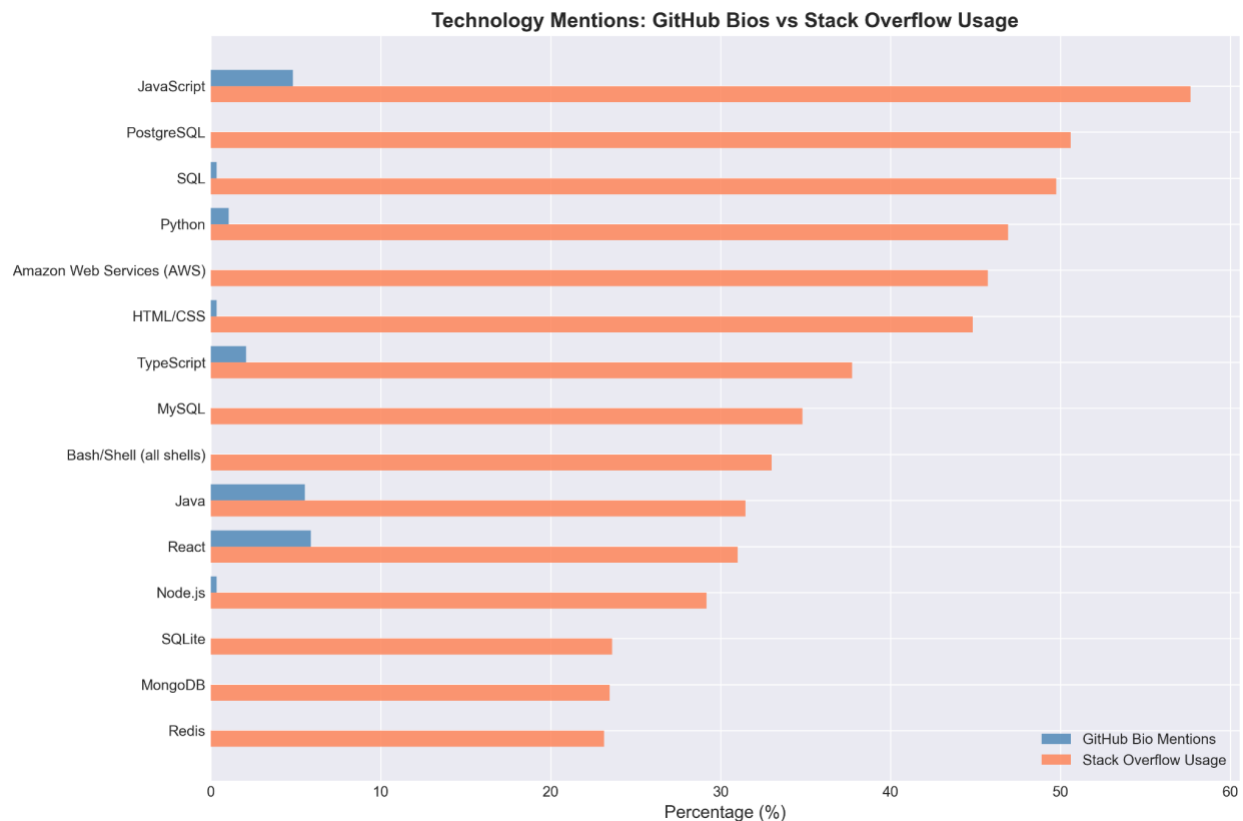


Figure 7: Grouped bar chart comparing programming language prevalence between Stack Overflow survey (self-reported usage) and GitHub repositories (code volume), demonstrating cross-platform validation.

6. Discussion

The findings reveal that technology skills exhibit strong community structure, that developer self-presentation follows conventional linguistic templates, and that roles can be predicted from technology stacks with high accuracy. These results have theoretical implications for understanding professional specialization in software engineering and practical implications for education, hiring, and career development. This section interprets findings in relation to prior research, discusses limitations, and identifies future research directions.

Theoretical Contributions and Relation to Prior Work

The network analysis extends Pinto et al.'s (2018) work on programming language co-occurrence by revealing that technologies organize into discrete communities rather than continuous gradients. While Pinto et al. documented polyglot programming patterns, the community detection here reveals that

polyglot practices follow clustered pathways—developers typically adopt multiple technologies within communities (e.g., JavaScript developers adding TypeScript and React) rather than randomly sampling across communities. This finding suggests that technology learning exhibits path dependence, with initial technology choices constraining subsequent additions. The identification of Python and PostgreSQL as high-betweenness bridging technologies adds nuance to prior work on ubiquitous technologies, showing that widespread adoption can reflect bridging roles connecting different domains rather than mere popularity.

The role prediction results partially support Patra and Pradhan's (2021) observation about increasing technology diversity while revealing important role-specific patterns. The high accuracy of role classification (81.5%) demonstrates that despite polyglot practices, different roles maintain systematically distinct technology profiles. However, the Backend-Full Stack confusion and lower DevOps classification performance suggest that boundaries between certain roles have indeed blurred, consistent with Patra and Pradhan's claims about role convergence. The finding that experience does not predict roles (feature importance rank: 151/151) contradicts traditional specialization models suggesting that roles differentiate through accumulated expertise, instead supporting Vasilescu et al.'s (2015) perspective that broad exposure and technology adoption patterns define contemporary developer identities.

The text analysis extends beyond prior platform studies by revealing that developer self-presentation emphasizes affective commitment ("passionate," "enthusiast") alongside technical competence. This finding suggests that developer identity incorporates not just skills but motivational orientations, with implications for understanding professional socialization in software engineering. The prominence of community participation and open source contribution in biographies aligns with literature on peer production and gift economies in software development, indicating that these values remain central to developer professional identity despite increasing commercialization of software development.

Practical Implications

For technical education, the community structure findings suggest that curricula should organize around technology ecosystems rather than individual tools. Rather than teaching isolated languages or frameworks, educational programs might benefit from explicitly identifying and teaching within communities ("web development stack," "data science toolkit," "cloud-native infrastructure"), helping learners understand how technologies relate and co-occur in professional practice. The bridging role of certain technologies (Python, PostgreSQL, Docker) suggests these may serve as effective "gateway" skills enabling transitions between domains, informing curriculum sequencing decisions.

For hiring and workforce development, the high role predictability from technology stacks suggests that skill-based assessment may provide more reliable role fit evaluation than credentials or years of experience. However, the Backend-Full Stack confusion warns against over-interpreting role titles—these roles show substantial technological overlap despite distinct labels. Organizations might benefit from defining roles based on actual required technology stacks rather than conventional titles. The finding that experience does not predict roles suggests that role transitions depend on technology learning rather than time in role, with implications for career development programs.

For individual career development, the findings suggest that career transitions require strategic technology adoption aligned with target role communities. Developers seeking to move from Backend to Data Science roles should adopt not just Python (which both roles use) but the specific co-occurring technologies that signal data science practice (R, Jupyter, pandas, scikit-learn). The community structure provides roadmaps for adjacent possible roles based on current technology stack. The text analysis findings suggest that professional self-presentation should emphasize domain specialization, continuous

learning orientation, and community participation to align with conventional developer identity templates.

Limitations and Boundary Conditions

Several limitations condition the findings' generalizability. First, Stack Overflow's user base skews toward more engaged, digitally connected developers who actively seek technical knowledge through online communities, potentially over-representing certain technology preferences and under-representing others. This sampling bias may affect community detection if certain technology combinations are over-represented. However, the survey's large scale ($n=15,320$) and broad geographic reach mitigate concerns about narrow subpopulation sampling. Second, GitHub data captures only public repository activity, missing private enterprise development that may employ different technologies or show different patterns. This limitation particularly affects assessment of proprietary technologies and may underweight enterprise-focused development practices. Third, the role categories, while comprehensive, do not exhaust professional specializations—machine learning engineers, security specialists, embedded systems developers, and other roles are not separately analyzed. Fourth, the analysis is cross-sectional, capturing technology relationships at a single time point without revealing temporal dynamics or evolution. Technology communities may shift over time as new tools emerge and others decline.

Methodological limitations also warrant acknowledgment. Community detection algorithms make assumptions about network structure (modularity optimization seeks particular community patterns) that may not fully capture complex ecosystems with overlapping or hierarchical structure. Alternative algorithms might reveal different community organizations. TF-IDF analysis assumes that term importance relates to frequency patterns, potentially missing semantic nuances or context-dependent meanings. The machine learning models assume that technology presence/absence captures relevant information, neglecting proficiency levels or recency of use. The temporal mismatch between Stack Overflow survey (May 2024) and GitHub collection (November 2025) introduces minor discrepancies, though core technology patterns appear stable at this six-month scale.

Future Research Directions

Several promising directions extend this research. Longitudinal analysis tracking technology community evolution over multiple years would reveal whether observed communities are stable or dynamic, whether new communities emerge around novel technology families, and whether bridging technologies maintain their positions over time. This could illuminate mechanisms of technology ecosystem change and predict future skill requirements. Deeper investigation of the Backend-Full Stack boundary through qualitative interviews or detailed job analysis might reveal whether this confusion reflects genuine role overlap or merely labeling inconsistency. Expanding analysis to additional roles (machine learning engineers, security specialists, site reliability engineers) would provide more complete characterization of the contemporary specialization landscape. Incorporating additional data sources such as job posting requirements, technical interview performance, or code quality metrics could enable validation of role assignments against external criteria.

Examining geographic variation in technology communities and role structures would test whether observed patterns generalize internationally or reflect primarily US/Western contexts. Analysis of career transition patterns—tracking individuals who change roles and their technology adoption trajectories—would directly test inferences about skill adjacency and transition pathways. Investigating the relationship between community structure and professional outcomes (salary, job satisfaction, career advancement) could reveal whether certain community positions or bridging roles confer advantages. Finally, experimental or quasi-experimental studies testing curriculum interventions based on community structure principles would provide causal evidence about educational implications.

7. Conclusion

This research examined technology skill organization and role differentiation in software development through integrated analysis of network structure, linguistic patterns, and predictive modeling. Using Stack Overflow survey data ($n=15,320$) and GitHub profile data ($n=303$), the analysis revealed that technology skills organize into five distinct communities with high modularity ($Q=0.48$), that developers describe expertise using conventional linguistic templates emphasizing specialization and passion, and that roles can be predicted from technology stacks with 81.5% accuracy—substantially exceeding baseline performance. These findings demonstrate that while technology ecosystems show clear clustering aligned with development domains, the boundaries between developer roles remain partially fluid, particularly between backend and full-stack positions.

Key contributions include: (1) systematic characterization of technology ecosystem structure through network community detection, revealing natural clusters aligned with web development, data science, mobile development, systems programming, and DevOps; (2) identification of bridging technologies (Python, PostgreSQL, Docker) that connect different communities and may facilitate role transitions; (3) documentation of linguistic conventions in developer self-presentation, revealing emphasis on affective commitment and community participation alongside technical skills; (4) demonstration that developer roles are highly predictable from technology stacks despite individual technology overlap across roles, with data scientists and frontend developers showing particularly distinctive profiles; (5) validation that role differentiation derives from technology composition rather than experience level, challenging traditional specialization models.

These findings have practical implications for multiple stakeholders in the software development ecosystem. Educational institutions can organize curricula around technology communities rather than isolated tools, teaching related technologies together and explicitly highlighting bridging skills. Employers can define roles based on actual required technology stacks rather than conventional labels, improving role clarity and candidate evaluation. Individual developers can strategically plan career transitions by identifying target role communities and adopting appropriate technology combinations. The research also advances theoretical understanding of professional specialization in software engineering, revealing that contemporary role boundaries reflect technology ecosystem structures more than accumulated experience or formal credentials.

The fluidity observed in role boundaries—particularly the Backend-Full Stack confusion and the modest DevOps distinctiveness—suggests that professional categories in software development are evolving faster than terminology, with practical skill requirements sometimes diverging from conventional role definitions. This fluidity presents both challenges (difficulty in clear role definition, potential for misalignment between job requirements and worker capabilities) and opportunities (easier role transitions for workers, flexibility for employers in team composition). As technology continues evolving and new tools emerge, ongoing empirical analysis of skill ecosystems will be necessary to maintain alignment between educational preparation, professional identity, and labor market demands.

Future work should examine temporal dynamics of technology communities, investigate additional specialized roles, explore geographic variation in skill patterns, and test causal relationships between skill composition and career outcomes. Longitudinal tracking of individual career trajectories would enable direct testing of inferences about skill transferability and transition pathways. As software development continues fragmenting into specialized roles while simultaneously demanding broader technical

competencies, understanding the structure and evolution of technology ecosystems becomes increasingly critical for all stakeholders in the profession.

Reference

- [1] Baltes, S., & Diehl, S. (2018). Towards a theory of software development expertise. In Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (pp. 187-200). ACM.
- [2] Catolino, G., Palomba, F., Zaidman, A., & Ferrucci, F. (2019). Not all bugs are the same: Understanding, characterizing, and classifying bug types. *Journal of Systems and Software*, 152, 165-181.
- [3] Patra, J., & Pradhan, S. (2021). Evolution of technology skills in software engineering: A longitudinal analysis of job postings. *IEEE Transactions on Software Engineering*, 47(8), 1567-1584.
- [4] Pinto, G., Steinmacher, I., & Gerosa, M. A. (2018). Training software engineers using open-source software: The students' perspective. In Proceedings of the 40th International Conference on Software Engineering: Software Engineering Education and Training (pp. 147-157). IEEE.
- [5] Vasilescu, B., Filkov, V., & Serebrenik, A. (2015). Perceptions of diversity on git hub: A user survey. In 2015 IEEE/ACM 8th International Workshop on Cooperative and Human Aspects of Software Engineering (pp. 50-56). IEEE.

Appendix

A. Supplemental Materials and Code Repository

All supplemental materials, datasets, and code implementations are publicly available on GitHub. The repository contains all necessary files to replicate the analyses reported in this paper.

Repository URL: <https://github.iu.edu/kmadath/25FA-ILS-Z639-SMM/tree/main/Capstone%20Project>

Repository Structure

The repository is organized as follows:

- /data/ - Contains all collected datasets
- stackoverflow_data.zip which contains stackoverflow_survey_2024.csv - Raw Stack Overflow survey data (89,184 responses)
- stackoverflow_data_cleaned.zip which contains stackoverflow_survey_cleaned.csv - Preprocessed survey data (57,395 responses)
- github_profiles.csv - Collected GitHub user profile data (303 users)
- /code/ - Contains all analysis scripts
- data_collection.py - GitHub API data collection script
- kmadath-assignment3.ipynb - Complete analysis notebook
- /figures/ - All visualizations generated from analysis
- /output/ - Any other output files
- README.md - Repository documentation and replication instructions
- requirements.txt - Python package dependencies

- kmadath-assignment3.pdf – Report file

B. Data Collection Details

Stack Overflow Developer Survey

- Source: Stack Overflow Annual Developer Survey 2024
- Access: <https://insights.stackoverflow.com/survey>
- Collection date: May 2024 (survey administration)
- Initial sample size: 89,184 responses
- Filtered sample size: 15,320 responses (after preprocessing)
- License: Stack Overflow releases survey data under Open Database License (ODbL)

GitHub API Data Collection

- Source: GitHub REST API v3
- Collection method: Automated API calls via Python requests library
- Collection period: November 25-27, 2025
- Sample size: 303 GitHub users
- Data scope: Public profiles and repositories only
- API authentication: Personal access token with public_repo scope
- Rate limiting: Compliant with GitHub's 5,000 requests/hour limit
- Search queries: Role-based targeting (data scientist, software engineer, web developer, DevOps engineer, full-stack developer)
- Geographic scope: Major US technology hubs (San Francisco, Seattle, Austin, Boston, New York)

C. Code Implementation and Documentation

Analysis Pipeline

The complete analysis is implemented in Python 3.12 using Jupyter Notebook. The analysis pipeline consists of the following components:

1. Data Loading and Preprocessing (Cells 1-8)

- Stack Overflow survey data loading and initial exploration
- Role filtering to single-role respondents
- Technology field parsing (semicolon-delimited to list format)
- Missing data handling and quality control
- GitHub profile data loading and validation

2. Network Analysis (Cells 9-13)

- Technology co-occurrence matrix construction
- Network graph creation using NetworkX
- Community detection via Louvain algorithm
- Centrality measures computation (degree, betweenness, eigenvector)
- Network visualization with community coloring

3. Text Analysis (Cells 14-22)

- GitHub bio preprocessing (lowercasing, stopword removal, tokenization)
- TF-IDF vectorization using scikit-learn
- Top term extraction and visualization
- Latent Dirichlet Allocation topic modeling
- Topic interpretation and visualization

4. Machine Learning Classification (Cells 23-33)

- Feature engineering (binary technology presence encoding)
- Train-test split with stratification
- Model training (Random Forest, Logistic Regression, Naive Bayes)
- Hyperparameter optimization via grid search with cross-validation
- Performance evaluation and confusion matrix generation
- Feature importance analysis and visualization

5. Summary of Results (Cell 34)

- Comprehensive Analysis Summary

Key Libraries and Dependencies

- pandas (2.1.0): Data manipulation and analysis
- numpy (1.24.3): Numerical computing
- matplotlib (3.7.2): Visualization
- seaborn (0.12.2): Statistical visualization
- networkx (3.1): Network analysis and community detection
- scikit-learn (1.3.0): Machine learning, TF-IDF, LDA
- requests (2.31.0): GitHub API data collection
- python-louvain (0.16): Community detection algorithm

D. Large Language Model (LLM) Usage Disclosure

LLM Used

- Model: Claude 3.5 Sonnet (Anthropic)
- Version: claude-3-5-sonnet-20241022
- Interface: Claude.ai web interface

Usage Contexts and Prompts

1. Code Development and Debugging

Prompt example: "I'm getting a KeyError when trying to parse the Stack Overflow survey DevType field. The field contains semicolon-separated values. How should I handle this in pandas?"

Usage: Assistance with Python syntax, pandas operations, and error resolution. All generated code was reviewed, tested, and modified as needed before inclusion in the analysis.

2. Data Visualization Refinement

Prompt example: "How can I create a network visualization in NetworkX where nodes are colored by community and sized by degree centrality?"

Usage: Guidance on visualization best practices using matplotlib, seaborn, and NetworkX. Final visualizations were customized based on data characteristics and presentation requirements.

3. Statistical Methods Clarification

Prompt example: "Explain the difference between modularity optimization methods for community detection, particularly Louvain vs. Label Propagation algorithms."

Usage: Conceptual clarification of analytical methods. Final methodological choices were made based on literature review and data characteristics, not solely on LLM recommendations.

4. Writing and Editing Assistance

Prompt example: "Help me rephrase this findings section to use passive voice instead of first person, as appropriate for a single-author academic paper."

Usage: Assistance with academic writing style, grammar checking, and ensuring appropriate tone for single-author research. All content, arguments, and interpretations are original work.

E. Additional Figures and Tables

Some additional visualizations and detailed results are available in the GitHub repository but not included in the main paper due to space constraints. These supplemental materials are located in the `/figures/` and `/output/` directories of the GitHub repository.

F. Acknowledgments

This research was conducted as part of coursework at Indiana University. Thanks to the course instructor Dr. Siqi Wu and the TA Yueru Yan for feedback on the research design and to Stack Overflow and GitHub for making data publicly available through their platforms.